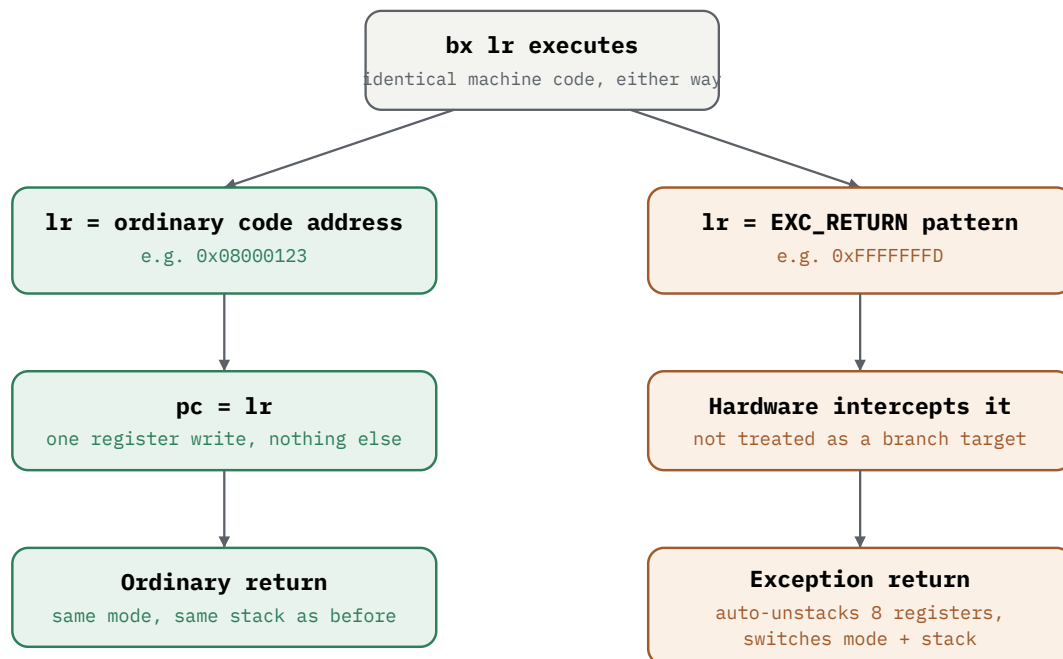


stm32f103 workspace notes

bx lr : a normal return vs. an exception return

Same mnemonic, same machine code, either way. The CPU doesn't tell these two cases apart by looking at the instruction - it tells them apart by looking at what's actually sitting in `lr` at that moment.



the fork isn't in the instruction - it's entirely in what value happened to be in `lr`

Every `EXC_RETURN` value lives in the `0xFFFFFFF0-0xFFFFFFFF` range - an address space that can never hold real code on a Cortex-M3, which is exactly what makes it safe to use as a signal rather than a genuine branch target.

What EXC_RETURN's bits actually mean

Only the low nibble carries information - the upper 28 bits are always the same fixed `0xFFFFFFF` pattern, purely so the value falls in that unimplementable range.

VALUE	RETURNS TO	USING
0xFFFFFFF1	Handler mode	MSP
0xFFFFFFF9	Thread mode	MSP

VALUE	RETURNS TO	USING
0xFFFFFFFF	Thread mode	PSP

0xFFFFFFFF is the exact value `vPortSVCHandler` forces with `orr r14, #0xd` - bit 2 set means "Thread mode," bit 0 set means "PSP."

Most handlers never touch lr at all

This is the detail that makes the whole scheme quietly elegant: hardware writes the correct `EXC_RETURN` value into `lr` automatically, the instant an exception is *entered* - based on whatever mode and stack were active right before it fired. A handler that does nothing special to `lr` will return correctly without knowing any of this happened.

two handlers, two very different relationships with `lr`

```
// SysTick_Handler - plain C, compiler-generated epilogue
// never touches lr - it's still exactly what hardware set at entry
void xPortSysTickHandler(void)
{
    /* ... */
} // compiler emits: bx lr (or pop {..., pc})
// this WORKS correctly with no special-casing at all

// vPortSVCHandler - naked, hand-written assembly
// FORCES a specific EXC_RETURN - it can't trust what's already there
    orr    r14, #0xd           ; overwrite whatever lr held -
    bx     r14                 ; force "Thread mode, PSP" regardless
```

Why the difference. `SysTick_Handler` is returning to *the same context that was interrupted* - whatever that was is still correct. `vPortSVCHandler` and `PendSV_Handler` exist specifically to hand off to a *different* task's context than whatever was running when the exception fired - "whatever was there before" is exactly the answer they can't use, so they overwrite `lr` deliberately before returning.

stm32f103 workspace notes · continues from the SysTick / PendSV / SVC and MSP notes